

Unity Catalog

**Common Patterns.
Clear Fixes.**

50 engineering patterns covering metastore design, grants and permissions, external locations, row and column filters, lineage and audit, volumes, and migration in Databricks Unity Catalog. Patterns every Unity Catalog engineer must recognise.

INTRODUCTION

Unity Catalog looks like governance. The gaps are where it is not.

Unity Catalog is Databricks' answer to the governance problem. A single metastore, a three-level namespace, fine-grained access controls, lineage tracking, and audit logging. It looks comprehensive from the documentation. The problems appear in production when grants do not behave as expected, when access controls have gaps that the UI does not surface, when migrations leave tables in inconsistent states, and when security controls that appear active are being bypassed through paths nobody audited.

This book covers seven chapters. Chapter 1 covers metastore design choices on Azure that are straightforward to get wrong and difficult to reverse. Chapter 2 covers grants and permissions and the specific ways Unity Catalog access controls fail silently or produce unexpected results, including the Azure identity boundary where Databricks-managed and Entra ID principals diverge. Chapter 3 covers external locations and the ADLS Gen2 RBAC paths that bypass Unity Catalog governance entirely if not closed. Chapter 4 covers row and column filters and the specific ways filters that pass every test can be bypassed. Chapter 5 covers lineage and audit and the patterns that do not generate Unity Catalog lineage or audit records. Chapter 6 covers managed and external volumes and the access failures they produce. Chapter 7 covers Hive metastore to Unity Catalog migration and the specific ways the migration breaks workloads when access connectors and managed identities are not set up first.

Every scenario follows the same structure: the situation, the wrong thinking, what is actually happening, and the fix. Unity Catalog problems are particularly consequential because they often involve security and compliance rather than just performance or correctness.

AZURE DATABRICKS ENGINEERING PATTERNS / BOOK 5

Chapter 1

Metastore Design

Seven scenarios covering the catalog and metastore topology decisions that are straightforward to get wrong and difficult to reverse once data and workloads are bound to them on Azure.

01

METASTORE DESIGN

You created one metastore for your entire organisation. Teams are fighting over catalog names.

THE SITUATION

A single Unity Catalog metastore is created for the entire Databricks organisation. Teams from different business units all share the same metastore. Within months, teams are in conflict over catalog naming conventions, one team accidentally dropped a catalog belonging to another team, and the metastore admin is a bottleneck for every governance change.

WRONG THINKING

Engineers create one metastore to keep the architecture simple and centralised. A single metastore seems like the right choice for a single organisation. The governance and coordination overhead of sharing a metastore across many independent teams is not anticipated.

WHAT IS ACTUALLY HAPPENING

A Unity Catalog metastore is the top-level governance boundary for all catalogs, schemas, and tables within it. Metastore admins have broad privileges including the ability to drop catalogs and manage all grants. When dozens of teams share a single metastore, the blast radius of an admin error or a misconfigured grant is the entire organisation's data. Catalog naming conflicts require coordination between teams that may have no relationship. A single metastore becomes a governance coordination problem that scales with the number of teams.

THE FIX

Design the metastore topology based on organisational and governance boundaries rather than technical convenience. For organisations with independent business units that have separate data governance requirements, separate metastores per business unit provide clean isolation. Within a business unit, a single metastore with well-governed catalogs per team or domain is appropriate. Limit metastore admin access to a small group of platform engineers and use catalog-level ownership for team-specific governance. Establish catalog naming standards before teams begin creating catalogs.

02

METASTORE DESIGN

You created a metastore per team. Cross-team queries are impossible.

THE SITUATION

Each team in the organisation has its own Unity Catalog metastore. Teams that need to query data owned by other teams find that cross-metastore queries are not supported. Sharing data between teams requires exporting and importing data, creating significant operational overhead.

WRONG THINKING

Engineers create separate metastores for each team to provide maximum isolation. Each team has full control over their metastore without interference from other teams. The need for cross-team data sharing is not considered during the initial design.

WHAT IS ACTUALLY HAPPENING

Unity Catalog does not support cross-metastore queries. A query running in workspace A attached to metastore A cannot directly reference tables in metastore B even if the user has permissions in both metastores. The three-level namespace `catalog.schema.table` is scoped to a single metastore. Cross-metastore data sharing requires Delta Sharing, data exports, or external table references rather than simple SQL joins.

THE FIX

Use a single metastore with catalog-level isolation rather than separate metastores per team for organisations where cross-team data sharing is required. Grant each team ownership of their own catalog within the shared metastore. Use catalog-level permissions to restrict what other teams can see and access within each team's catalog. Reserve separate metastores for cases where true organisational isolation is required such as different business units with separate compliance requirements or separate cloud subscriptions.

03

METASTORE DESIGN

Your Unity Catalog metastore is in a different region from your data. Query latency doubled.

THE SITUATION

A Unity Catalog metastore is created in one Azure region and Databricks workspaces with data stored in another Azure region are attached to it. Query latency for all queries against Unity Catalog tables increases significantly compared to the previous Hive metastore configuration.

WRONG THINKING

Engineers create the metastore in the most convenient region or the region where the Databricks account was first set up. The metastore region and the data region are treated as independent choices. The performance impact of cross-region metadata operations is not considered.

WHAT IS ACTUALLY HAPPENING

Unity Catalog metadata operations including table discovery, permission checks, and schema resolution require network calls to the metastore service. When the metastore is in a different region from the compute and data, every metadata operation incurs cross-region network latency. For queries with many table references, permission checks, or schema resolutions, the accumulated cross-region latency can be significant. This is particularly noticeable for interactive queries and dashboards where users expect sub-second response times.

THE FIX

Create the Unity Catalog metastore in the same Azure region as the primary Databricks workspaces and ADLS Gen2 storage accounts. For organisations with workspaces in multiple Azure regions, evaluate whether a single metastore with cross-region access or multiple region-specific metastores provides better performance given the expected cross-workspace data sharing requirements. Monitor Unity Catalog metadata operation latency as a separate metric from query execution time to identify cross-region overhead.

04

METASTORE DESIGN

You attached multiple workspaces to one metastore. A workspace admin deleted a shared catalog.

THE SITUATION

Multiple Databricks workspaces are attached to a single Unity Catalog metastore. A workspace admin in one workspace, attempting to clean up their workspace's resources, deletes a catalog that was shared across multiple workspaces. All teams using that catalog from other workspaces lose access to their data.

WRONG THINKING

Engineers attach multiple workspaces to a shared metastore and grant workspace admin permissions to team leads for their respective workspaces. Workspace admin is assumed to be scoped to the workspace. The extent of workspace admin privileges within a shared metastore is not reviewed before granting.

WHAT IS ACTUALLY HAPPENING

In Unity Catalog, workspace admins have significant privileges within the metastore including the ability to manage catalogs that are bound to their workspace. If a catalog is accessible from multiple workspaces and a workspace admin has CREATE or DROP privileges on that catalog, they can drop it even if other workspaces depend on it. The metastore does not prevent cross-workspace impact of privileged operations within a catalog.

THE FIX

Review the privilege model carefully before granting workspace admin to team leads in multi-workspace metastore configurations. Use catalog-level ownership rather than workspace admin to give teams control over their own catalogs without broader metastore privileges. Implement a catalog deletion protection policy that requires a separate approval step before any catalog DROP operation. Regularly audit which principals have DROP privileges on shared catalogs.

05

METASTORE DESIGN

Your three-level namespace is correct. Legacy code using two-level namespace is silently bypassing Unity Catalog.

THE SITUATION

Unity Catalog is enabled across all Databricks workspaces. The engineering team confirms all new code uses the three-level namespace `catalog.schema.table`. Legacy notebooks and jobs that use the two-level namespace `schema.table` continue to run successfully. A security audit reveals that the legacy code is accessing the Hive metastore directly rather than Unity Catalog, bypassing all Unity Catalog access controls and audit logging.

WRONG THINKING

Engineers confirm Unity Catalog is enabled and all new code uses the correct namespace. Legacy code using two-level namespace is assumed to be referencing Unity Catalog tables using the default catalog. The distinction between Hive metastore access and Unity Catalog access for two-level namespace references is not understood.

WHAT IS ACTUALLY HAPPENING

When a Unity Catalog-enabled workspace runs code that uses two-level namespace references, Databricks routes the query to the default catalog if one is configured, or to the Hive metastore if no default Unity Catalog catalog is set. Legacy code using `schema.table` syntax may be accessing the legacy Hive metastore databases rather than Unity Catalog catalogs depending on the cluster access mode and default catalog configuration. On clusters that are not configured for Unity Catalog shared or single-user access mode, two-level namespace references go to the Hive metastore entirely, bypassing Unity Catalog governance.

THE FIX

Audit all cluster access modes to confirm clusters are configured for Unity Catalog access modes rather than the legacy no-isolation shared mode. In Unity Catalog access modes, two-level namespace references use the configured default catalog. Configure a default catalog for all workspaces and clusters so two-level references resolve to Unity Catalog rather than the Hive metastore. Run a code audit to identify all notebooks and jobs using two-level namespace and update them to use three-level namespace explicitly.

06

METASTORE DESIGN

You migrated from Hive metastore to Unity Catalog. External tables lost their data.

THE SITUATION

A migration from the Hive metastore to Unity Catalog is performed using the Unity Catalog migration tools. After migration, several external tables show no data. Querying them returns empty results or errors. The underlying data files still exist in cloud storage.

WRONG THINKING

Engineers run the Unity Catalog migration tool and confirm the migration completed successfully. Tables appear in Unity Catalog with the correct schema. Empty results are assumed to be a query issue rather than a migration issue.

WHAT IS ACTUALLY HAPPENING

External tables in the Hive metastore reference data at a specific storage path. When migrated to Unity Catalog, the external table definition must reference an external location that covers the storage path. If the external location was not created before migration, or if the Azure Databricks access connector backing the external location does not have `Storage Blob Data Contributor` on the ADLS Gen2 container, Unity Catalog cannot read the data even though the table definition migrated correctly. The table exists in Unity Catalog but the storage access is broken.

THE FIX

Before running the Unity Catalog migration for external tables, create external locations that cover all ADLS Gen2 paths referenced by Hive metastore external tables. Verify that the access connector managed identity associated with each external location has at least `Storage Blob Data Contributor` on the storage account or container. After migration, run validation queries against each migrated external table to confirm data is accessible. Automate this validation as part of the migration process rather than relying on manual verification.

07

METASTORE DESIGN

Your catalog naming convention made sense at 3 teams. Is unmanageable at 30.

THE SITUATION

A Unity Catalog metastore is set up with catalogs named after team names: data_engineering, analytics, finance. As the organisation grows to 30 teams, catalog proliferation makes discovery difficult. Users cannot find the data they need and the metastore admin spends significant time explaining the catalog structure to new engineers.

WRONG THINKING

Engineers name catalogs after the team that owns them, which is a natural organisational mapping. As teams grow and split, each new team gets a new catalog. The discovery and navigation burden of 30 catalogs named after teams that may have changed since the catalog was created is not anticipated.

WHAT IS ACTUALLY HAPPENING

Catalog names that reflect organisational structure become stale as organisations evolve. Teams are renamed, merged, and split. A catalog named after a team that no longer exists or has a different name creates confusion. Engineers looking for financial data may not know which of several finance-related catalogs contains the data they need. Team-based naming also means data from the same domain is spread across multiple catalogs if different teams own parts of it.

THE FIX

Name catalogs after data domains rather than team names. Domain names such as customer, product, finance, and operations are stable even as team structures change. Within each domain catalog, use schemas to represent sub-domains or team ownership boundaries. Define a catalog naming standard before the metastore grows beyond 10 catalogs. Document the purpose and contents of each catalog in a data catalog or wiki so engineers can discover data without asking the metastore admin.

AZURE DATABRICKS ENGINEERING PATTERNS / BOOK 5

Chapter 2

Grants and Permissions

Eight scenarios covering the specific ways Unity Catalog access controls fail silently, produce unexpected results, or create gaps between what the permission UI shows and what users can actually do at runtime, including the Azure identity boundary.

08

GRANTS AND PERMISSIONS

You granted SELECT on a schema. Users cannot query any tables in it.

THE SITUATION

A data engineer grants SELECT on a schema to a group. Users in that group report they cannot query tables in the schema. They receive permission denied errors despite the grant appearing correct in the Unity Catalog permissions UI.

WRONG THINKING

Engineers grant SELECT at the schema level expecting it to propagate to all tables within the schema. Schema-level SELECT is assumed to work like a wildcard grant that covers all current and future tables.

WHAT IS ACTUALLY HAPPENING

Unity Catalog uses a hierarchical permission model where access to a securable requires USE permission on all parent objects. To query a table, a user needs USE CATALOG on the catalog, USE SCHEMA on the schema, and SELECT on the table or schema. Granting SELECT on the schema is necessary but not sufficient. If the users do not also have USE CATALOG on the parent catalog, they cannot access the schema even with SELECT granted. The permission denied error reflects the missing USE CATALOG grant rather than a missing SELECT grant.

THE FIX

When granting access to tables in Unity Catalog, always grant the full permission chain: GRANT USE CATALOG on the catalog, GRANT USE SCHEMA on the schema, and GRANT SELECT on the schema or individual tables. Use the Unity Catalog permission inheritance model by granting at the schema level for broad access and at the table level for selective access. After granting, validate access by testing a query from a user in the target group before marking the access provisioning as complete.

09

GRANTS AND PERMISSIONS

You granted ALL PRIVILEGES to a user. They still cannot create tables.

THE SITUATION

A user is granted ALL PRIVILEGES on a catalog. The user reports they cannot create tables in schemas within the catalog. The grants page shows ALL PRIVILEGES granted to the user.

WRONG THINKING

Engineers grant ALL PRIVILEGES expecting it to cover all possible operations including table creation. ALL PRIVILEGES is assumed to be the highest permission level that unlocks every capability.

WHAT IS ACTUALLY HAPPENING

ALL PRIVILEGES in Unity Catalog grants the set of privileges applicable to that securable object. For a catalog, ALL PRIVILEGES includes USE CATALOG, CREATE SCHEMA, and other catalog-level privileges, but it does not automatically propagate to schemas within the catalog. To create a table in a schema, the user also needs CREATE TABLE privilege on the schema. ALL PRIVILEGES on the catalog grants catalog-level privileges but not schema-level privileges. The user can create schemas but cannot create tables in existing schemas unless they also have CREATE TABLE on those schemas.

THE FIX

Grant privileges at the appropriate level for the required operation. For a user who needs to create tables in existing schemas, grant CREATE TABLE on each schema explicitly or grant ALL PRIVILEGES on the schema rather than just the catalog. For a user who needs broad access across all objects in a catalog, grant ALL PRIVILEGES at both the catalog level and the schema level. Review the Unity Catalog privilege hierarchy documentation to understand which privileges apply at each level before setting up complex permission structures.

10

GRANTS AND PERMISSIONS

Your permission grants look correct. Users get access denied at runtime.

THE SITUATION

A Unity Catalog permission setup appears correct in the grants UI. All required grants are visible for the user's group. Users from that group still receive access denied errors when running queries against the tables.

WRONG THINKING

Engineers verify the grants in the Unity Catalog UI and confirm all necessary permissions appear to be granted. When runtime access denied errors still occur, the grants are assumed to be cached incorrectly or to have a Databricks platform bug.

WHAT IS ACTUALLY HAPPENING

Unity Catalog permissions apply to the principal specified in the grant. If a user is a member of a group and the grant is on the group, the user inherits the grant. However if the group membership was changed after the user's current session started, the new membership may not be reflected in the running session. Additionally if grants were made to a group but the user was added to the group after the grant was created, the grant applies to the group including the new member, but session-level permission caching may not have refreshed.

THE FIX

Ask users experiencing permission issues to restart their cluster or start a new session to refresh the permission cache. Verify group membership is correct and that the user appears in the group that has the relevant grants. Check whether the grants were made to a nested group rather than directly to the group the user belongs to. Unity Catalog resolves nested group membership but this resolution has a propagation delay. Use the Unity Catalog permission checker tool or run a permission validation query as the affected user to confirm what permissions are actually visible in the current session.

11

GRANTS AND PERMISSIONS

You revoked access from a user. They can still query the table through a shared cluster.

THE SITUATION

A user's access to a sensitive table is revoked in Unity Catalog. The user continues to query the table successfully from a shared all-purpose cluster. The revocation appears correct in the Unity Catalog grants view.

WRONG THINKING

Engineers revoke the Unity Catalog grant and assume access is immediately and completely removed. The revocation is considered complete once the grants UI shows the permission is removed.

WHAT IS ACTUALLY HAPPENING

Unity Catalog permission enforcement depends on the cluster access mode. On clusters configured with Unity Catalog Single User or Shared access mode, Unity Catalog permissions are enforced for every query. On legacy clusters using No Isolation Shared access mode, Unity Catalog permission enforcement does not apply. In that mode, queries access data through whatever credentials the cluster itself holds, which on Azure is the cluster's managed identity or a service principal configured via Spark configuration. A user on a no-isolation shared cluster retains access to data files in ADLS Gen2 even after their Unity Catalog grant is revoked, as long as the cluster's underlying Azure identity still has `Storage Blob Data Reader` or higher on the relevant storage account.

THE FIX

Audit all clusters for access mode configuration. Disable or restrict the No Isolation Shared access mode for workloads involving sensitive data; account admins can disable it workspace-wide via the account console's Disable legacy features setting. Ensure all clusters used to access governed data run in Single User (Dedicated) or Shared access mode, both of which enforce Unity Catalog permissions for every query. After revoking access, verify the revocation is effective by asking the user to run a test query from a correctly configured cluster rather than any available cluster. As a defense in depth, also remove direct ADLS Gen2 RBAC grants from end users so that even a no-isolation cluster cannot bypass Unity Catalog.

12

GRANTS AND PERMISSIONS

Your service principal has the right grants. Jobs are still failing with permission errors.

THE SITUATION

A service principal is granted the necessary Unity Catalog permissions to run production jobs. The grants are verified in the Unity Catalog UI. Jobs running under this service principal fail with permission denied errors on specific tables.

WRONG THINKING

Engineers verify the service principal's grants and confirm they cover the tables being accessed. Since the grants look correct, the permission errors are assumed to be a Databricks platform issue. The distinction between a Databricks-managed service principal, a Microsoft Entra ID service principal, and the managed identity of the underlying Azure resource is not investigated.

WHAT IS ACTUALLY HAPPENING

Databricks jobs run as the principal configured in the job task's run-as setting. On Azure there are three distinct identity types that can be in play: a Databricks-managed service principal (workspace-local), a Microsoft Entra ID service principal federated into the Databricks account, and the managed identity (system-assigned or user-assigned) attached to the job's compute. Unity Catalog grants must be on the identity that actually runs the job. If a job is configured to run as a Databricks service principal but the underlying compute uses a different Entra ID managed identity for storage access, the Unity Catalog query check passes against the Databricks SP while the storage access goes through the managed identity, or vice versa. Audit logs in `system.access.audit` show whichever identity the access actually used.

THE FIX

Confirm the exact identity under which the job runs by checking the job's Run as setting in the workflow definition and the cluster's identity configuration. Verify Unity Catalog grants are on the same Entra ID object ID that `system.access.audit` shows is performing the operations. Add an identity verification step at the start of each job: `spark.sql('SELECT current_user(), session_user()').show()` logs the active identity. Prefer Entra ID service principals federated into the Databricks account over workspace-local Databricks service principals, since the former give you one consistent identity across compute, storage, and Unity Catalog.

13

GRANTS AND PERMISSIONS

You granted access to a view. Users can see underlying table data through the view definition.

THE SITUATION

A view is created over a sensitive table to expose only specific columns. Users are granted SELECT on the view but not on the underlying table. Users discover they can see the view definition including the full table name and column names by running DESCRIBE EXTENDED on the view.

WRONG THINKING

Engineers use views as a security layer to restrict column access. Granting access only to the view and not the underlying table is assumed to prevent users from knowing anything about the underlying data structure.

WHAT IS ACTUALLY HAPPENING

DESCRIBE EXTENDED and SHOW CREATE TABLE on a view reveal the view definition including the SQL query used to create it. If the view definition contains table names, column names, or filter conditions, users with SELECT on the view can infer significant information about the underlying table structure even without direct table access. For sensitive data where the existence of certain columns or the table structure itself is confidential, views do not hide the schema from users who can inspect view definitions.

THE FIX

For use cases where the underlying table structure must be hidden from view consumers, use dynamic views with function-based column masking rather than simple column selection views. Additionally restrict DESCRIBE and SHOW CREATE TABLE access on views containing sensitive definition logic if needed. For column-level security where the goal is to hide specific data values rather than column existence, use Unity Catalog column masks which apply at the table level and do not expose their implementation through view definitions.

14

GRANTS AND PERMISSIONS

Group membership grants are working. Direct user grants are being ignored.

THE SITUATION

A user is granted `SELECT` directly on a table as an individual user grant. The user also belongs to a group that has been explicitly denied access to the same table. The user receives access denied errors even though they have a direct grant.

WRONG THINKING

Engineers assume a direct user grant overrides a group-level deny. Individual grants are expected to take precedence over group-level restrictions because they are more specific.

WHAT IS ACTUALLY HAPPENING

Unity Catalog uses an additive permission model where privileges accumulate from all applicable grants including user grants, group grants, and inherited grants. However explicit `DENY` grants, if supported in the configuration, override positive grants. More commonly, the issue arises when a group has no access to a parent object such as a catalog or schema, which prevents the user from exercising their table-level grant even though the table grant exists. If the group lacks `USE CATALOG` and the user's individual grant chain does not include a separate `USE CATALOG` grant, the table grant is unreachable.

THE FIX

Audit the complete permission chain for affected users by checking grants at the catalog, schema, and table level for both the user directly and all groups the user belongs to. Ensure the user has a complete permission chain through either individual grants or group membership that covers `USE CATALOG`, `USE SCHEMA`, and `SELECT`. Avoid complex scenarios where individual user grants conflict with group denials as these are difficult to reason about and maintain. Prefer managing access entirely through groups to keep the permission model predictable.

15

GRANTS AND PERMISSIONS

You granted permissions in Unity Catalog. They are not reflected in the Hive metastore.

THE SITUATION

After setting up Unity Catalog grants, a legacy notebook running on a cluster in legacy access mode queries a table using the Hive metastore path. The query succeeds despite the user not having Unity Catalog grants for that table. The Unity Catalog grants appear to have no effect on Hive metastore access.

WRONG THINKING

Engineers configure Unity Catalog grants expecting them to govern all data access including data that was previously governed by the Hive metastore. Unity Catalog is treated as a single governance layer that supersedes all other access controls.

WHAT IS ACTUALLY HAPPENING

Unity Catalog governs access only for queries running through Unity Catalog-enabled clusters in the appropriate access modes. The Hive metastore has its own separate permission model based on database-level grants. Tables in the Hive metastore can still be accessed through legacy cluster configurations that bypass Unity Catalog entirely. Unity Catalog grants do not retroactively apply to Hive metastore objects or to queries running through legacy access modes. The two permission systems are independent.

THE FIX

Migrate all data from the Hive metastore to Unity Catalog and deprecate Hive metastore access to ensure a single governance layer. Restrict or eliminate the use of legacy cluster access modes that bypass Unity Catalog. Monitor Hive metastore access in audit logs and alert when it is used in production workloads that should be governed by Unity Catalog. Establish a migration timeline for all legacy workloads and enforce it through cluster access mode restrictions.

AZURE DATABRICKS ENGINEERING PATTERNS / BOOK 5

Chapter 3

External Locations

Seven scenarios covering the gaps between what external location configuration appears to enforce and what it actually allows, including the ADLS Gen2 RBAC paths that bypass Unity Catalog governance entirely if not closed.

16

EXTERNAL LOCATIONS

You created an external location. Users can still access the storage directly bypassing Unity Catalog.

THE SITUATION

An external location is created in Unity Catalog to govern access to an ADLS Gen2 container. Access is restricted to specific groups. Users without Unity Catalog access to the external location continue to access files at the storage path directly using the Azure Storage SDK, `az storage` CLI commands, or `dbutils.fs` on legacy clusters.

WRONG THINKING

Engineers create external locations expecting them to be the enforcement point for all access to the governed storage path. An external location is assumed to work like a firewall that blocks all direct storage access.

WHAT IS ACTUALLY HAPPENING

External locations in Unity Catalog govern access through Databricks compute to cloud storage. They do not modify the Azure RBAC role assignments on the underlying storage account that govern direct ADLS Gen2 API access. If the storage account or container has `Storage Blob Data Reader` or higher assigned to end users (either directly or via an Entra ID group), those users can bypass Unity Catalog external location controls entirely by authenticating directly to ADLS Gen2 with their own Entra ID credentials. The Microsoft Learn documentation explicitly calls this out: granting users, service principals, or managed identities direct access to ADLS Gen2 containers that contain Unity Catalog data bypasses Unity Catalog and exposes the data to over-permissioning and exfiltration.

THE FIX

Unity Catalog external locations on Azure must be paired with a locked-down storage account RBAC model. Remove all `Storage Blob Data *` role assignments from end users and end-user groups on storage accounts that back Unity Catalog external locations. The only identities that should hold `Storage Blob Data Contributor` on those accounts are the Azure Databricks access connectors used as storage credentials. For high-sensitivity data, additionally enable the storage account firewall and configure trusted access for the access connector's managed identity so that even leaked end-user Entra ID credentials cannot reach the data plane. External location governance is only complete when storage account RBAC and network controls are aligned with Unity Catalog governance.

17

EXTERNAL LOCATIONS

Your external location is defined correctly. `dbutils.fs.ls` returns permission denied.

THE SITUATION

An external location is created and granted to the appropriate groups. Users with the correct Unity Catalog grants attempt to use `dbutils.fs.ls` to list files at the external location path and receive permission denied errors.

WRONG THINKING

Engineers create the external location, set up grants, and assume users can immediately use all file operations at the path including directory listings with `dbutils.fs`. The external location grant is assumed to enable all file-level operations.

WHAT IS ACTUALLY HAPPENING

`dbutils.fs` operations go through Databricks File System APIs which have their own access checks separate from Unity Catalog table-level access. Access to an external location through Unity Catalog enables SQL-level access to tables defined over that location. Direct file system operations using `dbutils.fs` on external location paths require the user to have READ FILES or WRITE FILES privilege on the external location specifically, not just access to tables built on that location. A user who can query a table at the external location path may not be able to list the directory or read raw files.

THE FIX

Grant READ FILES on the external location explicitly for users who need direct file system access using `dbutils.fs` or file-level APIs. `GRANT READ FILES ON EXTERNAL LOCATION location_name TO group_name`. For write operations, grant WRITE FILES separately. Review whether direct file system access is necessary for each use case, since table-level access through SQL is the preferred access pattern for governed data and avoids the need for raw file system permissions.

18

EXTERNAL LOCATIONS

Two external locations overlap in path. Access to both is behaving unexpectedly.

THE SITUATION

Two external locations are created where one covers a parent path and another covers a subdirectory of the same parent. Users with access to the parent external location can access the subdirectory even though the subdirectory external location was intended to restrict access to a smaller group.

WRONG THINKING

Engineers create a more specific external location to restrict access to a sensitive subdirectory within a broader external location. The subdirectory external location is expected to override the parent external location for that path.

WHAT IS ACTUALLY HAPPENING

Unity Catalog external location path matching follows a longest-prefix match rule. For a given storage path, Unity Catalog applies the most specific matching external location. However if a user has READ FILES on the parent external location, they can access any path under that parent regardless of whether a more specific external location exists for a subdirectory, because the parent grant covers the full path hierarchy. The subdirectory external location restricts who can create a new external location or external table at that path, but it does not revoke access granted by a parent external location.

THE FIX

Avoid overlapping external location paths where a parent path and a subdirectory path are both configured as external locations with different access controls. To restrict access to a sensitive subdirectory, move it to a completely separate storage path that is not a subdirectory of any other external location. This ensures path isolation at the storage level rather than relying on Unity Catalog path matching semantics to enforce access restrictions.

19

EXTERNAL LOCATIONS

You deleted an external location. Tables pointing to it became unreadable silently.

THE SITUATION

An external location that is no longer actively used is deleted during a cleanup exercise. Several days later, downstream pipelines begin failing with external location not found errors. Investigation reveals that tables defined over the deleted external location's path are now unreadable.

WRONG THINKING

Engineers delete an external location that appears unused based on the absence of recent activity in the Unity Catalog UI. Tables built on the external location are not surfaced in the UI as dependents of the external location before deletion.

WHAT IS ACTUALLY HAPPENING

External tables in Unity Catalog reference storage paths that must be covered by an external location. When an external location is deleted, all external tables whose storage paths fall under that location's path become inaccessible because Unity Catalog can no longer validate and provide access to the underlying storage. The tables still exist as metadata objects in the catalog but all data access fails. Unity Catalog does not prevent deletion of external locations that have dependent tables, and the UI does not show dependent tables as a deletion warning by default.

THE FIX

Before deleting any external location, query the Unity Catalog system tables to identify all external tables whose location paths fall under the external location path. Run `SELECT * FROM system.information_schema.tables WHERE table_type = 'EXTERNAL' AND location LIKE 'storage_path%'` to find all dependent tables. Migrate or drop these tables before deleting the external location. Implement a pre-deletion check as a mandatory step in any external location lifecycle management process.

20

EXTERNAL LOCATIONS

Your storage credential expired. Every job touching external storage failed simultaneously.

THE SITUATION

All Databricks jobs that access external storage through Unity Catalog external locations fail simultaneously with credential expired or authentication failed errors. Investigation reveals that the client secret on the Entra ID service principal used by every storage credential expired overnight and the new secret was not updated in Unity Catalog.

WRONG THINKING

Engineers set up storage credentials once during initial configuration using an Entra ID service principal client secret, and assume they are managed by Entra ID and remain valid indefinitely. The Entra ID credential expiration policy is not reflected in the Unity Catalog configuration.

WHAT IS ACTUALLY HAPPENING

Unity Catalog storage credentials store the authentication information used to access cloud storage on behalf of Databricks. On Azure, a storage credential can be backed by either an Entra ID service principal with a client secret or by an Azure Databricks access connector with a managed identity. Service principal client secrets have configurable maximum lifetimes enforced by Entra ID (24 months maximum, often shorter under organisational policies). When the secret expires, every storage credential bound to that service principal becomes invalid and all external locations using those credentials fail at the same moment. Managed identities have no such secret because Azure platform handles the trust relationship directly.

THE FIX

Migrate every Azure storage credential to use an Azure Databricks access connector with a system-assigned or user-assigned managed identity rather than an Entra ID service principal client secret. Managed identities have no rotation cycle because there is no secret to expire. Microsoft Learn explicitly recommends this pattern for Unity Catalog on Azure. If service principal credentials must be retained for some specific reason, monitor secret expiration via Entra ID expiration alerts, automate the rotation with `ALTER STORAGE CREDENTIAL credential_name SET credential ...`, and distribute credentials across multiple storage credential objects rather than using a single credential for all external locations to reduce the blast radius of an expiry.

21

EXTERNAL LOCATIONS

You created an external location in one workspace. It is not visible in another workspace on the same metastore.

THE SITUATION

An external location is created and validated in workspace A. A team in workspace B, which is attached to the same Unity Catalog metastore, cannot see the external location when they try to create an external table using the same storage path.

WRONG THINKING

Engineers create external locations at the metastore level and expect them to be visible in all workspaces attached to the metastore. Since Unity Catalog is a metastore-level construct, all metastore resources are assumed to be available in all attached workspaces.

WHAT IS ACTUALLY HAPPENING

External locations are metastore-level objects visible in all workspaces attached to the metastore. However visibility does not equal access. A user in workspace B cannot use an external location unless they have been granted USE permission on that external location or are a metastore admin. Additionally the ability to create external tables or volumes at the external location path requires CREATE EXTERNAL TABLE or CREATE EXTERNAL VOLUME privilege on the external location. Users in workspace B see the external location only if they have metastore admin privileges or have been explicitly granted access.

THE FIX

Grant the appropriate privileges on the external location to the groups that need to use it from other workspaces. GRANT CREATE EXTERNAL TABLE ON EXTERNAL LOCATION location_name TO group_name for groups that need to create tables. GRANT READ FILES for groups that need file-level access. External locations are shared governance objects, not workspace-specific resources, so access management must cover all workspaces and principals that legitimately need access.

22

EXTERNAL LOCATIONS

Your external location validates successfully. Spark reads from it return empty results.

THE SITUATION

An external location is created and the validate button in the Unity Catalog UI shows it is working correctly. External tables created over this location are queryable through SQL but return empty results. A Spark notebook reading directly from the path using `spark.read` also returns an empty DataFrame.

WRONG THINKING

Engineers run the external location validation and accept a successful result as confirmation that the location is correctly configured and contains accessible data. Empty query results are investigated as a data quality issue rather than a configuration issue.

WHAT IS ACTUALLY HAPPENING

The external location validation check verifies that Unity Catalog can authenticate to the storage path using the storage credential. It confirms that the credential has sufficient permissions to list the directory and perform basic file operations. It does not verify that the path contains any data files or that data files at the path match the expected format. A successfully validated external location over an empty directory or a directory containing files in an unexpected format will pass validation but return empty results.

THE FIX

After validating the external location, independently verify that the expected data files exist at the path using a direct listing operation. Use `dbutils.fs.ls('storage_path')` or the cloud provider's storage console to confirm files are present. If files are present and queries still return empty results, check that the file format specified in the external table definition matches the actual file format at the path. Validate the schema against the actual file content using an inference read before creating the external table.

04

AZURE DATABRICKS ENGINEERING PATTERNS / BOOK 5

Chapter 4

Row and Column Filters

Seven scenarios covering the specific ways Unity Catalog row filters and column masks that pass every test can be bypassed, produce unexpected performance, or fail in production contexts.

23

ROW AND COLUMN FILTERS

Your row filter is applied. Users are seeing rows they should not through an unfiltered view.

THE SITUATION

A row filter is applied to a sensitive table restricting access to rows relevant to each user based on their group membership. The filter works correctly when users query the table directly. A view is created over the table without any additional filters. Users query the view and see all rows regardless of the row filter.

WRONG THINKING

Engineers apply a row filter to the table and assume it applies to all access paths to that table including queries through views. A row filter on the table is expected to be inherited by any view built on it.

WHAT IS ACTUALLY HAPPENING

Unity Catalog row filters apply to direct table access. When a view is created over a filtered table, the view definition is executed as the view owner or definer, not as the current user in all configurations. If the view owner has broader access than the row filter would permit for the querying user, the view may bypass the row filter. Additionally some view configurations execute the underlying table query with the definer's permissions rather than the invoker's permissions, meaning the row filter based on `CURRENT_USER()` evaluates to the view owner rather than the actual querying user.

THE FIX

Apply row filters directly to views as well as to the underlying tables when views are used as access layers. Alternatively use invoker's rights views which execute the underlying query as the current user rather than the view owner, ensuring that row filters based on current user identity are correctly applied. Review all views built on row-filtered tables to confirm they do not create unintended bypass paths. Test access through each view as a user who should be filtered and verify only permitted rows are returned.

24

ROW AND COLUMN FILTERS

Column masking is configured. The masked values appear in query results via a join.

THE SITUATION

A column mask is applied to a PII column in a customer table, replacing the actual email address with a masked version for users without PII access. A user without PII access runs a join between the masked customer table and another table that contains the same email addresses in an unmasked column. The join condition reveals the unmasked email values from the other table that correspond to the masked customer records.

WRONG THINKING

Engineers apply column masking to protect PII in the customer table. The masking is validated by querying the customer table directly and confirming masked values appear. Protection of the masked column is assumed to prevent PII exposure through all query patterns.

WHAT IS ACTUALLY HAPPENING

Column masking replaces sensitive values in query results from the masked table with the masked version. However if the same data exists in another table without masking, a join can expose the original values through the join output from the unmasked table. Column masking protects the column in the table where it is defined. It does not prevent correlation of masked values with unmasked values from other data sources that a user has access to.

THE FIX

Column masking is one layer of PII protection but it is not sufficient when the same data appears in multiple tables. Apply consistent data classification and masking across all tables containing the same PII values. Use row-level or column-level access controls on the unmasked table to prevent users without PII access from joining against it. Conduct a data inventory to identify all tables that contain the same PII values and apply consistent governance across all of them.

25

ROW AND COLUMN FILTERS

Your row filter uses `CURRENT_USER()`. It works interactively. Breaks in job context.

THE SITUATION

A row filter uses `CURRENT_USER()` to restrict rows to data relevant to the querying user. Interactive queries from notebooks work correctly, returning only the rows appropriate for each user. When a Databricks job runs a query against the filtered table, the job fails or returns empty results because the job's service principal identity does not have any rows associated with it in the filter logic.

WRONG THINKING

Engineers implement user-based row filtering using `CURRENT_USER()` and validate it interactively. The filter works correctly for all tested user identities. Job execution is not tested before the filter is deployed to production.

WHAT IS ACTUALLY HAPPENING

Row filters using `CURRENT_USER()` evaluate to the identity executing the query. For interactive users this is a named user identity that exists in the filter lookup table. For jobs running as service principals, `CURRENT_USER()` evaluates to the service principal's identity, which typically does not have a corresponding entry in the filter lookup table. The row filter returns no rows for the service principal because no filter rules exist for that identity. The job receives an empty result set rather than a useful error.

THE FIX

Design row filters to handle both user identities and service principal identities. Add service principal identities to the filter lookup table with appropriate row access definitions when jobs need to access filtered tables. Alternatively implement row filter logic that returns all rows for service principal identities used in production jobs while applying user-based filtering only for interactive user sessions. Test row filter behavior with service principal identities before deploying to production.

26

ROW AND COLUMN FILTERS

Row filters are slowing every query by 10x regardless of how many rows are filtered.

THE SITUATION

A row filter is added to a large Delta table to restrict access by region. After the filter is applied, all queries against the table are 10 times slower than before the filter was added, regardless of whether the filter eliminates most rows or just a few.

WRONG THINKING

Engineers assume a row filter that eliminates most rows should make queries faster by reducing the data scanned. A filter that returns 1 percent of rows should be 100 times faster, not 10 times slower.

WHAT IS ACTUALLY HAPPENING

Unity Catalog row filters are implemented as subquery conditions appended to every query against the table. If the filter function is complex or references external data such as a lookup table containing group memberships, the filter subquery runs for every query regardless of the query's own filtering conditions. A filter that joins against a group membership table adds a join to every query. If the group membership table is not optimised, the added join dominates query time regardless of how few rows ultimately pass the filter.

THE FIX

Profile the row filter function in isolation by running it directly as a query and measuring its execution time. Optimise the filter's data sources: if the filter references a group membership table, ensure it is cached, small, and well-indexed. Consider materialising group memberships as a broadcast-eligible DataFrame rather than a live table join. If the filter cannot be made fast, evaluate whether column masking or a separate governed view provides better performance for the specific access control requirement.

27

ROW AND COLUMN FILTERS

You applied a column mask. The mask is bypassed when users query the Delta files directly.

THE SITUATION

A column mask is applied to a sensitive column in a Delta table. Users accessing the table through SQL in Databricks notebooks and jobs see the masked values correctly. A power user with storage access reads the Delta Parquet files directly from cloud storage using a local Spark session or a Python script with the cloud SDK. They see the unmasked values in the raw files.

WRONG THINKING

Engineers apply column masking and validate it works for SQL queries in Databricks. Column masking is assumed to protect the data universally since it is applied at the Unity Catalog level.

WHAT IS ACTUALLY HAPPENING

Unity Catalog column masks are query-layer controls enforced by the Databricks compute engine when queries pass through Unity Catalog governance. The actual data in the Delta Parquet files is stored unmasked on ADLS Gen2. Anyone with `Storage Blob Data Reader` or higher on the storage account containing the masked table can read the raw Parquet files via the Azure Storage SDK, the `az storage` CLI, or Azure Storage Explorer and see the unmasked values without going through Unity Catalog. Column masking protects access through governed Databricks compute but does not encrypt or modify the physical data in storage.

THE FIX

For data that must be protected against direct file access, implement encryption at the storage layer (Customer-Managed Keys on the storage account, or column-level encryption applied before write) or physically mask the data in the stored files via a transformation pipeline, rather than relying solely on Unity Catalog query-layer masking. Restrict ADLS Gen2 access at the RBAC layer to only the Azure Databricks access connector managed identity used by the external location, and remove all direct `Storage Blob Data *` role assignments from end users. Treat Unity Catalog column masking as a query-layer convenience rather than a complete security control for highly sensitive data.

28

ROW AND COLUMN FILTERS

Your row filter function references another table. That table is inaccessible to some users.

THE SITUATION

A row filter references a group membership lookup table to determine which rows each user can access. Some users report they receive errors when querying the filtered table rather than filtered results. Investigation reveals these users do not have SELECT access to the lookup table referenced by the row filter.

WRONG THINKING

Engineers implement a row filter that references a lookup table and test it with users who have access to both the filtered table and the lookup table. Testing passes. Users without access to the lookup table are not tested before deployment.

WHAT IS ACTUALLY HAPPENING

Unity Catalog row filter functions are executed in the context of the querying user. If the filter function references a table, the querying user must have SELECT access to that referenced table for the filter to execute successfully. When a user without access to the lookup table queries the filtered table, the row filter function fails with a permission error rather than silently returning an empty result set. The user's inability to access the lookup table causes the filter itself to fail, making the primary table inaccessible.

THE FIX

Grant SELECT access to the row filter lookup table to all users who need to query the filtered table, even if they should not otherwise have access to the lookup table's contents for other purposes. Alternatively restructure the filter to use built-in functions that do not require external table access, such as using Databricks session variables or user attribute functions that return group memberships without requiring a table lookup. Test row filter behavior with users who have the minimum expected access level before deploying to production.

29

ROW AND COLUMN FILTERS

Column masking works for SELECT queries. Returns unmasked values in DESCRIBE queries.

THE SITUATION

A column mask replaces sensitive values with masked equivalents for users without PII access. Direct SELECT queries return masked values correctly. A user without PII access runs DESCRIBE TABLE EXTENDED on the masked table and can see the unmasked default values or sample values that appear in the column statistics stored in the Delta table metadata.

WRONG THINKING

Engineers validate column masking by running SELECT queries and confirming masked values appear. DESCRIBE operations are administrative metadata queries assumed to be governed by the same column masking controls as data queries.

WHAT IS ACTUALLY HAPPENING

Unity Catalog column masks apply to data returned by SELECT queries. DESCRIBE TABLE EXTENDED and similar metadata inspection commands return table metadata including column statistics such as min, max, and null count values that are computed from the actual unmasked data and stored in the Delta table metadata. These statistics may reveal information about the unmasked data range. Users who cannot see data values through SELECT may be able to infer value ranges from metadata statistics.

THE FIX

For tables with column masks on sensitive data, evaluate whether column statistics in the Delta table metadata need to be restricted. Consider running ANALYZE TABLE without computing statistics for sensitive columns, or clearing existing statistics using table property settings. This prevents metadata inspection from revealing statistical information about masked columns. For highly sensitive data, test all access paths including DESCRIBE, SHOW STATISTICS, and information schema queries to identify any metadata leakage beyond SELECT queries.

AZURE DATABRICKS ENGINEERING PATTERNS / BOOK 5

Chapter 5

Lineage and Audit

Six scenarios covering the specific data sources and write patterns that do not generate Unity Catalog lineage or audit records, creating gaps between what governance reports show and what actually occurred.

30

LINEAGE AND AUDIT

Your Unity Catalog lineage shows a table was read. The job that read it no longer exists.

THE SITUATION

Unity Catalog lineage shows that a sensitive table was read by a specific job. When the security team investigates the access, they find that the job was deleted and the user who owned it has left the company. The lineage record exists but the access cannot be investigated further.

WRONG THINKING

Engineers review lineage to audit data access and assume the lineage record provides sufficient context to investigate any access. A lineage entry showing a job read a table is assumed to be enough information for audit purposes.

WHAT IS ACTUALLY HAPPENING

Unity Catalog lineage records the fact that a compute entity read a table, capturing the job ID and timestamp. The lineage record is a point-in-time snapshot. If the job is subsequently deleted, the lineage record remains but the job configuration, notebook code, and audit trail for what the job did with the data are no longer accessible. Lineage shows that access occurred but not what was done with the data after access.

THE FIX

Retain job run history and notebook versioning beyond the default retention periods for tables containing sensitive data. Export job metadata and run logs to a durable store as part of job execution. Implement a data access audit log that captures not just what was read but what workload read it, preserving enough context for post-incident investigation. Treat the deletion of jobs and notebooks that accessed sensitive data as requiring an access review before deletion is permitted.

31

LINEAGE AND AUDIT

Lineage is not captured for tables written using Spark DataFrameWriter directly.

THE SITUATION

A pipeline writes data to a Unity Catalog table using

```
df.write.saveAsTable('catalog.schema.table')
```

Unity Catalog lineage does not show any lineage for the written table. The table appears as an island with no upstream lineage despite being produced by a well-defined pipeline.

WRONG THINKING

Engineers write to Unity Catalog tables using standard Spark DataFrame APIs and expect Unity Catalog to automatically capture lineage for all writes. Lineage capture is assumed to be transparent regardless of the write API used.

WHAT IS ACTUALLY HAPPENING

Unity Catalog lineage capture depends on how queries and writes are executed. SQL-based operations and Delta Live Tables operations generate lineage automatically. Direct Spark DataFrame writes using the DataFrameWriter API may not generate Unity Catalog lineage in all Databricks Runtime versions and configurations. Lineage capture for programmatic writes requires specific runtime versions and may require the Unity Catalog lineage feature to be enabled and the operation to flow through specific API paths.

THE FIX

Use SQL-based writes such as `spark.sql('INSERT INTO catalog.schema.table SELECT ...')` or `spark.sql('CREATE OR REPLACE TABLE catalog.schema.table AS SELECT ...')` for operations where lineage capture is important. These SQL paths reliably generate Unity Catalog lineage. For existing DataFrameWriter-based pipelines, upgrade to a Databricks Runtime version that supports lineage capture for programmatic writes and verify lineage is generated after upgrading. Check the Databricks Runtime release notes for the specific version where DataFrameWriter lineage capture was enabled.

32

LINEAGE AND AUDIT

Your audit logs show a user accessed sensitive data. The user says they did not.

THE SITUATION

Unity Catalog audit logs show that a specific user accessed a sensitive table at a time when the user says they were not working. The user's account credentials appear to have been used but the user denies the access.

WRONG THINKING

Engineers use Unity Catalog audit logs as definitive evidence of user access. The audit log is trusted as an authoritative record of which user accessed which data.

WHAT IS ACTUALLY HAPPENING

Unity Catalog audit logs record the identity that was used for each access, not necessarily the human who authorised it. If a user's personal access token was used in an automated job rather than their interactive session, the audit log shows the user's identity even though the access was from an automated process. Additionally if credentials were leaked and used by an unauthorised party, the audit log correctly shows the access but the legitimate user was not involved. The audit log reflects the identity used, which may not be the person associated with that identity.

THE FIX

Investigate credential usage patterns when audit log records conflict with user statements. Check whether the user's access token is used in any automated jobs or scripts by searching the workspace for the token in job configurations. Implement a PAT token rotation policy and short expiry periods for interactive tokens to limit the window of exposure for leaked credentials. Require service principals for all automated access rather than personal access tokens to make audit logs more accurately reflect human versus machine access.

33

LINEAGE AND AUDIT

Column-level lineage is missing for tables written from Python UDFs.

THE SITUATION

A pipeline uses Python UDFs to transform data and writes the result to a Unity Catalog table. Unity Catalog shows table-level lineage indicating the output table was produced from the input table. Column-level lineage showing which input columns produced which output columns is absent.

WRONG THINKING

Engineers deploy a pipeline with Python UDFs and expect Unity Catalog to capture column-level lineage automatically, showing the full transformation from source to target column.

WHAT IS ACTUALLY HAPPENING

Unity Catalog column-level lineage is derived from SQL query plans. The Databricks lineage engine can trace column dependencies through SQL operations including joins, projections, and aggregations because the SQL query plan explicitly records column transformations. Python UDFs are opaque to the lineage engine. When a Python UDF takes multiple input columns and produces an output column, the lineage engine cannot determine which input columns contributed to the output without executing and inspecting the Python code. Column-level lineage is not captured for UDF-transformed columns.

THE FIX

For pipelines where column-level lineage is required, replace Python UDFs with built-in Spark SQL functions or Pandas UDFs that the lineage engine can trace. Where Python UDFs are necessary and column-level lineage is important, document the column transformation logic manually in a data catalog or metadata store. Consider adding explicit column lineage annotations to the pipeline code as comments or metadata that a separate lineage documentation process can capture.

34

LINEAGE AND AUDIT

Lineage shows correct upstream tables. Downstream consumers are not tracked.

THE SITUATION

Unity Catalog lineage shows the upstream source tables that feed a particular output table. The downstream consumers of that output table are not shown in the lineage graph. When the engineering team needs to assess the impact of changing the output table schema, they have no way to identify which downstream jobs and dashboards will be affected.

WRONG THINKING

Engineers use Unity Catalog lineage to perform impact analysis before schema changes. Lineage showing upstream sources is assumed to also capture downstream consumers.

WHAT IS ACTUALLY HAPPENING

Unity Catalog lineage is captured when queries execute against the metastore. Downstream lineage showing which jobs read a table is captured when those jobs run and their queries are recorded. If downstream consumers have not run recently, or if they read the table through methods that do not generate lineage such as direct Parquet reads or legacy cluster access modes, they will not appear in the downstream lineage graph. Lineage is a record of observed access, not a declaration of all intended consumers.

THE FIX

Supplement Unity Catalog lineage with a data contract or dependency registry where downstream consumers explicitly register their dependency on upstream tables. When planning a schema change, query both the Unity Catalog lineage for recent consumers and the dependency registry for declared consumers. Run a workspace-wide search for the table name in notebook code and job definitions to identify consumers that may not have run recently enough to appear in lineage. Treat lineage as a starting point for impact analysis, not a complete picture.

35

LINEAGE AND AUDIT

Your audit log retention is set to 90 days. A compliance audit needs 2 years of history.

THE SITUATION

A regulatory compliance audit requires demonstrating data access history for the past 2 years. Unity Catalog audit logs are configured with 90-day retention. Only 90 days of access history is available. The compliance team cannot satisfy the audit requirement.

WRONG THINKING

Engineers configure 90-day audit log retention as a reasonable operational choice. The 90-day window is sufficient for incident investigation. The compliance requirement for 2 years of audit history is not communicated to the platform engineering team when the retention is configured.

WHAT IS ACTUALLY HAPPENING

Unity Catalog audit log retention is a configurable setting in the Databricks account console. The default retention may not match compliance requirements for regulated industries. Once logs are purged beyond the retention window they are permanently lost. Retroactively extending retention does not recover logs that were already deleted.

THE FIX

Establish audit log retention requirements from compliance and legal teams before configuring the platform rather than using defaults. For regulated industries, 2 to 7 year retention is commonly required. The `system.access.audit` system table is queryable directly from Unity Catalog, but for retention beyond the platform default, configure Databricks audit log delivery to an ADLS Gen2 storage account in your subscription with a separate retention policy independent of the Databricks platform. Daily delivery to ADLS Gen2 ensures logs survive even if the Databricks workspace is decommissioned, and the storage account can be configured with immutability policies (legal hold or time-based) for regulated retention.

06

AZURE DATABRICKS ENGINEERING PATTERNS / BOOK 5

Chapter 6 Volumes

Seven scenarios covering the operational differences between managed and external volumes, the access failures they produce, and the specific ways volume file operations differ from cloud storage file operations.

36

VOLUMES

You created a managed volume. Files written to it are not visible through the storage account.

THE SITUATION

A managed volume is created in Unity Catalog. Files are written to the volume using `dbutils.fs` and confirmed to exist by listing the volume path. A team attempting to access these files directly through the cloud storage account UI or API cannot find them in any expected location.

WRONG THINKING

Engineers create a managed volume expecting it to be a governed wrapper around a regular cloud storage path. Files written to the volume are assumed to be accessible through both Unity Catalog and direct storage access using the same path.

WHAT IS ACTUALLY HAPPENING

Managed volumes store files in a Unity Catalog-managed storage location that is internal to Databricks. The exact storage path is managed by Databricks and is not a predictable user-accessible cloud storage path. Unlike external volumes which are backed by a user-specified external location, managed volumes use Databricks-controlled storage. Direct access to managed volume files through the cloud storage console requires knowing the Databricks-managed internal path, which is not the same as the volume's logical path in Unity Catalog.

THE FIX

Use external volumes when direct cloud storage access is required alongside Unity Catalog governance. External volumes are backed by a user-specified external location path that is accessible through both Unity Catalog and direct storage APIs. Use managed volumes when Databricks-managed storage and Unity Catalog-only access is acceptable and when portability of the storage path is not required. Document the difference between managed and external volumes in team onboarding materials to prevent confusion about storage accessibility.

37

VOLUMES

Your external volume path conflicts with an existing external location.

THE SITUATION

An external volume is created at a path that overlaps with an existing external location. The external volume creation appears to succeed but subsequent operations on the volume behave unexpectedly, with some operations succeeding and others failing with path conflict errors.

WRONG THINKING

Engineers create an external volume at a convenient path in existing cloud storage without checking whether the path conflicts with existing external location definitions.

WHAT IS ACTUALLY HAPPENING

Unity Catalog external locations and external volumes share the governance of cloud storage paths. When an external volume is created at a path that falls under an existing external location, both the external location and the volume have governance claims on the same path. Operations that go through the external location may succeed while operations specifically routed through the volume may fail, or vice versa, depending on which governance path is used for each operation. The conflict creates ambiguous governance that Unity Catalog may not resolve consistently.

THE FIX

Check for existing external locations that cover the intended volume path before creating an external volume. Use a path that is exclusively dedicated to the volume rather than a subdirectory of an existing external location. If the storage structure requires the volume to be within a path already covered by an external location, delete or modify the external location to exclude the volume path before creating the volume.

38

VOLUMES

You granted READ VOLUME. Users can list files but cannot read their contents.

THE SITUATION

READ VOLUME is granted to a group on an external volume. Users in that group can successfully list the files in the volume using `dbutils.fs.ls`. Attempts to read the file contents using `spark.read` or `dbutils.fs.head` fail with permission denied errors.

WRONG THINKING

Engineers grant READ VOLUME expecting it to enable full read access to the volume including both directory listing and file content reading.

WHAT IS ACTUALLY HAPPENING

READ VOLUME in Unity Catalog grants access to list the volume and use it as a source in SQL and Spark operations through Unity Catalog governed paths. The READ FILES privilege on the underlying external location may be separately required for certain low-level file read operations. Additionally, reading file contents through Spark requires the external location's storage credential to be accessible for the read operation, which requires the reading identity to have appropriate external location privileges beyond just volume-level grants.

THE FIX

When file content access is failing despite READ VOLUME being granted, also grant READ FILES on the external location backing the external volume. This provides access at both the volume governance layer and the storage credential layer. Test both directory listing and file content reading after granting permissions to confirm both operations work as expected. For volumes where only SQL and Spark table reads are required, READ VOLUME alone may be sufficient, but document which operations each privilege level enables.

39

VOLUMES

Files written to a volume are not appearing in Unity Catalog lineage.

THE SITUATION

A pipeline writes model artifacts and configuration files to a Unity Catalog volume. When reviewing Unity Catalog lineage for the pipeline, the volume write is not shown as a downstream lineage node.

WRONG THINKING

Engineers use volumes for storing ML artifacts and configuration files and expect the volume writes to appear in Unity Catalog lineage alongside table writes, providing a complete picture of pipeline outputs.

WHAT IS ACTUALLY HAPPENING

Unity Catalog lineage tracking is primarily designed for structured data assets such as tables and views. File writes to volumes are not currently tracked as lineage events in Unity Catalog's lineage graph. The lineage graph shows table-to-table and table-to-view relationships derived from SQL query plans. File writes to volumes go through file system APIs that do not generate lineage events in the same way SQL operations do.

THE FIX

For pipeline outputs that include both tables and volume files, document the volume write dependencies separately through a pipeline metadata registry or documentation layer. Use Unity Catalog tags on volumes to associate them with the pipelines that write to them, providing a manual lineage link. Monitor for Databricks announcements regarding volume lineage support as this capability may be added in future Unity Catalog releases.

40

VOLUMES

Your volume is used for ML model storage. Models saved there are not versioned by MLflow.

THE SITUATION

A Unity Catalog volume is used to store serialised ML model files. Models are saved to the volume path directly. The MLflow model registry does not show these models as registered versions. The team cannot use MLflow model lifecycle management for models stored this way.

WRONG THINKING

Engineers store ML models in Unity Catalog volumes expecting the combination of Unity Catalog governance and MLflow to provide integrated model management. Storing files in a Unity Catalog volume is assumed to integrate with MLflow model versioning.

WHAT IS ACTUALLY HAPPENING

Unity Catalog volumes provide governed file storage but they do not integrate with MLflow model versioning automatically. MLflow model versioning requires models to be logged through the MLflow APIs using `mlflow.log_model` or registered through the MLflow model registry. Simply writing serialised model files to a volume path does not create MLflow model versions, run metadata, or model signatures. The models are stored as files but are invisible to the MLflow model registry.

THE FIX

Log models to the Unity Catalog MLflow model registry using `mlflow.log_model` with the Unity Catalog model registry path as the destination. MLflow natively supports Unity Catalog as a model registry backend using the three-level namespace for model names. This provides both Unity Catalog governance and MLflow versioning. If serialised model files must also be stored in volumes for other reasons, save them separately to the volume after logging through MLflow rather than instead of it.

41

VOLUMES

You deleted a volume. The underlying files in external storage were also deleted.

THE SITUATION

An external volume that is no longer needed is deleted from Unity Catalog. The data engineering team later discovers that the actual files in the cloud storage location backing the external volume have also been deleted. The files contained reference data that was not backed up.

WRONG THINKING

Engineers assume that deleting an external volume is a metadata operation that removes the Unity Catalog governance layer but leaves the underlying files in external storage intact. Deleting the external volume is treated as safe since external storage is assumed to be independent.

WHAT IS ACTUALLY HAPPENING

The behavior of file deletion when an external volume is dropped depends on the DROP operation used and the Unity Catalog version. Some DROP VOLUME operations include file deletion in addition to metadata removal, particularly when the DROP command is issued with a cascade option or when the volume's contents are considered owned by Unity Catalog. The distinction between DROP VOLUME and DROP VOLUME CASCADE and their respective effects on underlying storage is not always clearly communicated in error messages or UI dialogs.

THE FIX

Before dropping any volume, verify whether the drop operation will also delete underlying files by checking the Unity Catalog documentation for the specific version and DROP syntax being used. For external volumes, prefer revoking access and marking the volume as deprecated rather than dropping it immediately. When a drop is necessary, back up the underlying files to a separate location first. Test DROP VOLUME behavior in a staging environment with non-critical data before applying it to production volumes containing irreplaceable data.

42

VOLUMES

Your volume path is correct. `dbutils.fs.cp` to a volume fails with an unsupported operation error.

THE SITUATION

Files are copied to a Unity Catalog volume using `dbutils.fs.cp`. The volume path is confirmed correct and the user has WRITE FILES access. The copy operation fails with an unsupported operation or not implemented error.

WRONG THINKING

Engineers use `dbutils.fs.cp` as the standard file copy tool in Databricks and expect it to work with Unity Catalog volume paths as it does with other storage paths.

WHAT IS ACTUALLY HAPPENING

Unity Catalog volumes are accessed through a specific API layer that enforces governance controls. Not all `dbutils.fs` operations are supported on volume paths in all Databricks Runtime versions. Operations like `dbutils.fs.cp`, `dbutils.fs.mv`, and `dbutils.fs.mkdirs` may have different support levels for volume paths compared to cloud storage paths. The unsupported operation error indicates the specific `dbutils` operation has not been implemented for the volume API layer.

THE FIX

Use the Databricks Files API or Python `shutil` operations for file management within volumes. For copying files to volumes, use `with open('/Volumes/catalog/schema/volume/file', 'wb') as f: f.write(data)` or use the Databricks REST API files endpoint. Check the Databricks documentation for the specific Runtime version being used to identify which file operations are supported for volume paths. As Databricks Runtime versions advance, additional file operations are being added to the volume API layer.

AZURE DATABRICKS ENGINEERING PATTERNS / BOOK 5

Chapter 7

Migration

Eight scenarios covering the specific ways migrating from the Hive metastore to Unity Catalog breaks workloads, loses data, or creates inconsistent state on Azure, including the access connector and managed identity setup that must precede the migration.

43

MIGRATION

You upgraded to Unity Catalog. All existing notebooks using `spark.table` broke.

THE SITUATION

Unity Catalog is enabled across all workspaces. Existing notebooks that use `spark.table('schema.table')` begin failing with table not found errors. The tables exist in Unity Catalog using the three-level namespace.

WRONG THINKING

Engineers enable Unity Catalog and assume existing two-level namespace table references will automatically resolve to the equivalent Unity Catalog tables. The migration is expected to be transparent to existing code.

WHAT IS ACTUALLY HAPPENING

`spark.table('schema.table')` resolves using the active session catalog. In a Unity Catalog-enabled workspace on a cluster in Unity Catalog access mode, if no default catalog is set or if the default catalog is different from where the tables were migrated, the two-level reference does not resolve to the expected Unity Catalog table. The Hive metastore used the default `spark_catalog`. After Unity Catalog migration, the catalog context changes and two-level references may resolve to the wrong catalog or fail to resolve at all.

THE FIX

Set a default catalog for all workspaces using the Unity Catalog workspace configuration so two-level references resolve to the expected catalog. Update notebooks to use three-level namespace references for all table access. Run a workspace-wide code search for `spark.table` calls with two-level references and update them to include the catalog name. Deploy this update as part of the Unity Catalog migration process before migrating tables to avoid a period where code breaks.

44

MIGRATION

Your Hive metastore tables migrated to Unity Catalog. External tables lost their location.

THE SITUATION

External tables are migrated from the Hive metastore to Unity Catalog using the migration tool. After migration the tables appear in Unity Catalog with their schema intact. Queries against the migrated external tables fail with location not found or access denied errors.

WRONG THINKING

Engineers run the Unity Catalog migration tool and see the tables appear in the Unity Catalog UI with correct schemas. The migration is considered complete. Subsequent query failures are attributed to configuration issues rather than migration incompleteness.

WHAT IS ACTUALLY HAPPENING

External table migration copies the table metadata including the ADLS Gen2 storage path to Unity Catalog. However for the external table to be queryable in Unity Catalog, the storage path must be covered by an external location with appropriate storage credentials. If external locations were not created before migration, or if the Azure Databricks access connector backing those external locations does not have `Storage Blob Data Contributor` on the relevant container, the table metadata migrated successfully but the storage access required to read the table did not.

THE FIX

Create external locations covering all Hive metastore external table paths before running the migration tool. Verify that the access connector managed identity associated with each external location has at least `Storage Blob Data Contributor` on the storage account or container. After migration, run a validation query against every migrated external table before declaring the migration complete. Include external location setup as a required pre-migration step in the migration runbook.

45

MIGRATION

You enabled Unity Catalog on a workspace. Legacy cluster access modes stopped working.

THE SITUATION

Unity Catalog is enabled on a workspace that has many existing clusters configured with the legacy no-isolation shared access mode. After enabling Unity Catalog, jobs running on these clusters start failing. Some fail with access denied errors and others fail because they cannot access Hive metastore tables using their previous paths.

WRONG THINKING

Engineers enable Unity Catalog as a workspace setting and expect existing clusters to continue working unchanged. Unity Catalog is expected to be additive, enabling new capabilities without breaking existing ones.

WHAT IS ACTUALLY HAPPENING

Unity Catalog enablement changes how cluster access modes interact with the Hive metastore and table resolution. Clusters in no-isolation shared mode may lose the ability to create new tables in the Hive metastore or may have changed behavior for two-level namespace resolution after Unity Catalog is enabled. Some operations that previously ran on no-isolation clusters fail on Unity Catalog-enabled workspaces because the security context enforcement changes. Not all cluster configurations are compatible with Unity Catalog-enabled workspaces.

THE FIX

Before enabling Unity Catalog on a workspace with existing workloads, audit all active clusters for their access mode and test critical workloads in a Unity Catalog-enabled staging workspace first. Migrate workloads from no-isolation shared clusters to Unity Catalog shared or single-user access mode clusters as part of the enablement process. Plan the enablement during a maintenance window and have a rollback plan ready. Do not enable Unity Catalog on production workspaces without a tested migration plan for all existing cluster configurations.

46

MIGRATION

Migrated tables show correct schema. Data quality checks are failing on migrated data.

THE SITUATION

Tables are migrated from the Hive metastore to Unity Catalog. The migrated tables show the correct schema in the Unity Catalog UI. Existing data quality checks that previously passed against the Hive metastore tables now fail against the migrated Unity Catalog tables despite the data being identical.

WRONG THINKING

Engineers verify the schema of migrated tables and confirm it matches the original. With the same schema and the same data, data quality checks are expected to produce the same results.

WHAT IS ACTUALLY HAPPENING

Data quality check failures after migration can be caused by subtle differences in how Unity Catalog and the Hive metastore handle certain data types, null values, or string encodings. If the migrated tables are external tables, the storage format and path must be identical to what the Hive metastore referenced. If the migration created managed tables by copying data rather than migrating the external table reference, the data copy may have introduced encoding differences or type casting behavior differences between the two metastores.

THE FIX

Run side-by-side queries against the original Hive metastore table and the migrated Unity Catalog table for a sample of the data to identify specific rows or columns where values differ. Check whether the migration created a managed table copy or migrated the external table reference, and verify the storage path is identical if it should be an external table reference. For managed table migrations, compare the row counts, checksum values, and statistical distributions of key columns between source and target to identify any data transformation that occurred during the copy.

47

MIGRATION

Your Unity Catalog migration succeeded. Third party BI tools cannot connect anymore.

THE SITUATION

Tables are migrated from the Hive metastore to Unity Catalog. Databricks SQL queries work correctly. Third-party BI tools such as Tableau, Power BI, and Looker that were connecting to the workspace using JDBC or ODBC drivers stop returning data or lose their table connections after the migration.

WRONG THINKING

Engineers validate the migration through Databricks SQL and notebook queries which work correctly. Third-party tool connections are assumed to work automatically since the same data is accessible through Databricks compute.

WHAT IS ACTUALLY HAPPENING

Third-party BI tools connecting through JDBC or ODBC drivers may use two-level namespace references stored in their connection configurations or visualisation definitions. After migration to Unity Catalog, these two-level references may resolve to the Hive metastore rather than Unity Catalog, or may fail to resolve entirely depending on the JDBC driver version and workspace configuration. Additionally Unity Catalog introduces a three-level namespace that older driver versions may not support or may not handle correctly in catalog browsing.

THE FIX

Update BI tool connection configurations to use the three-level namespace for all table references. Update JDBC and ODBC drivers to versions that support Unity Catalog three-level namespace. Set a default catalog in the connection configuration so that two-level references resolve correctly for tools that cannot be updated to use three-level references. Test all BI tool connections as part of the migration validation process before declaring the migration complete.

48

MIGRATION

You ran the Unity Catalog migration tool. Some tables were silently skipped.

THE SITUATION

The Unity Catalog migration tool is run to migrate all Hive metastore tables. The tool reports completion without errors. A post-migration audit reveals that several tables are still in the Hive metastore and were not migrated. The tool provided no warning about the skipped tables.

WRONG THINKING

Engineers trust the migration tool completion message as confirmation that all eligible tables were migrated. Tables that were not migrated are expected to be reported as errors or warnings if they could not be migrated.

WHAT IS ACTUALLY HAPPENING

The Unity Catalog migration tool skips tables that do not meet migration requirements without always raising explicit errors. Tables may be skipped if they use unsupported SerDe formats, if their storage paths are not covered by any existing external location, if they have certain table properties that are incompatible with Unity Catalog, or if they reference storage in regions that do not match the metastore region. The tool prioritises completing the migration for eligible tables and silently skips ineligible ones in some configurations.

THE FIX

After running the migration tool, run a reconciliation query comparing the list of tables in the Hive metastore against the list of tables in Unity Catalog to identify any that were not migrated. Investigate each unmigrated table to understand why it was skipped. Address the root cause for each category of skipped tables: create external locations for tables with uncovered storage paths, update incompatible table properties, or convert unsupported table formats before re-running the migration for those tables.

49

MIGRATION

Post-migration, DESCRIBE TABLE shows correct metadata. SELECT returns wrong data types.

THE SITUATION

Tables are migrated to Unity Catalog and DESCRIBE TABLE shows the expected schema including correct column names and data types. Running SELECT queries returns values where certain columns have unexpected types such as strings instead of integers or timestamps instead of dates.

WRONG THINKING

Engineers validate the migration by checking DESCRIBE TABLE which shows the correct schema. With the correct schema confirmed, query results are expected to match the declared types.

WHAT IS ACTUALLY HAPPENING

DESCRIBE TABLE shows the declared schema stored in the metastore metadata. The actual data in the underlying files may have been written with different types that do not exactly match the declared schema. Hive metastore tables often use schema on read where the declared type is used to cast the raw file data at query time. After migration to Unity Catalog, if the casting behavior differs between the Hive metastore reader and the Unity Catalog reader, the same underlying files produce differently typed query results. This is particularly common for date and timestamp types which have different default behaviors across Spark versions.

THE FIX

Investigate type mismatches by reading the underlying Parquet files directly without any schema enforcement to see the raw physical types stored in the files. Compare the raw physical types against the declared schema. For each mismatched column, determine whether the conversion behavior changed between the Hive metastore and Unity Catalog reader and update the downstream pipeline expectations accordingly. For critical columns, consider casting explicitly in the query rather than relying on implicit schema-on-read behavior.

50

MIGRATION

Your Unity Catalog rollout broke existing job service principals across all workspaces.

THE SITUATION

Unity Catalog is rolled out across multiple workspaces simultaneously. After the rollout, all jobs using service principals begin failing with authentication or permission errors. The service principals worked correctly before the rollout.

WRONG THINKING

Engineers roll out Unity Catalog as a workspace-level configuration change and assume service principals that were working before will continue working without modification.

WHAT IS ACTUALLY HAPPENING

Unity Catalog introduces new permission requirements for the identities used to run jobs. On Azure, a service principal that had implicit access through Hive metastore database grants or through the cluster's underlying managed identity for direct ADLS Gen2 access may no longer have access after Unity Catalog is enabled, because Unity Catalog requires explicit grants at the catalog, schema, and table level. An Entra ID service principal that previously accessed a Hive metastore table through a cluster's managed identity now needs explicit Unity Catalog grants on the equivalent Unity Catalog table, plus federation of that service principal into the Databricks account. The rollout changes the access model and the service principals have not been re-provisioned for the new model.

THE FIX

Before rolling out Unity Catalog to production workspaces, inventory all Entra ID service principals and the resources they access. Federate each service principal into the Databricks account so it can be granted Unity Catalog privileges. For each service principal, determine the equivalent Unity Catalog grants required (USE CATALOG, USE SCHEMA, SELECT/MODIFY) and provision them before the rollout. Test service principal access in a staging workspace with Unity Catalog enabled before applying to production. Roll out Unity Catalog to one workspace at a time rather than all simultaneously to limit blast radius and allow for iterative permission provisioning.

CONCLUSION

Governance is what the cluster will actually enforce.

Across 50 scenarios the pattern repeats. Unity Catalog grants look correct in the UI while the cluster access mode bypasses them. External locations enforce SQL-layer governance while storage account RBAC remains wide open. Row filters protect the table while a join from another table leaks the values. Column masks work for SELECT while DESCRIBE EXTENDED exposes the statistics. Migration tools report success while silently skipping incompatible tables. The lesson is that Unity Catalog is a governance layer, not the only layer. Real governance requires aligning Unity Catalog grants, cluster access modes, ADLS Gen2 RBAC, and storage network controls so that no path to the data exists outside the one Unity Catalog observes.

ALSO IN THE SERIES

Clusters and Compute

You enabled Spot VMs to cut cost. Jobs now randomly fail at 9am.

Delta Lake

Your MERGE is creating duplicate rows despite a unique match condition.

Structured Streaming and Auto Loader

Auto Loader processed the same file twice. Duplicate rows appeared downstream.

And more. Download the full series free at ssanjaychandra.com

A NOTE ON THIS RELEASE

This is Book 5 in the Azure Databricks Engineering Patterns series. The Unity Catalog mechanics described here apply on any cloud, with Azure-specific context introduced where the platform genuinely changes the answer: Azure Databricks access connectors with managed identities as the recommended storage credential, ADLS Gen2 RBAC and the storage account firewall as the controls that must be aligned with Unity Catalog external locations, and Entra ID service principal federation as the identity model for production jobs. Databricks and Azure both evolve quickly and some guidance may need updating as the platforms mature. If you spot a technical inaccuracy or a scenario that deserves deeper treatment, your feedback is genuinely welcome.

ACKNOWLEDGEMENTS

Claude (Anthropic) was used as a writing assistant while putting this book together.

CONNECT



www.ssanjaychandra.com